

Employee API

Full Improvement Roadmap

Handwritten notes · English · EmployeeAPICRUD

Status: planned, unless a checklist box is already ticked.

How to use these notes

- Build one phase at a time. Do not jump ahead.
- If an API contract changes, update three places: tests, the README, and employee-ui.

Why this file lives here

Best home: EmployeeAPICRUD/docs/ROADMAP.md

- Docs sit with the backend module.
- README stays short: run steps and setup only.

Avoid putting this whole plan inside README.md.

The README gets long and hard to scan.

Repo-root docs/ is better only when several modules share the same documents.

Avoid .cursor/plans/ for this roadmap.

That folder is Cursor-internal. A poor fit for GitHub readers and a portfolio.

Current baseline – already built

- Layered flow: controller → service → repository
- DTOs: *EmployeeRequest* and *EmployeeResponse*
- Bean Validation is already on those DTOs
- CORS allows local UI on ports 5173 and 3000
- Unit tests and Web MVC tests are in place
- *EmployeeNotFoundException* → global handler → 404

Request path (remember this)

Client

- *EmployeeController* /api/v1
- *EmployeeService*
- *EmployeeRepository* Spring Data JPA
- MySQL *employeedb.employee*

Gaps this roadmap closes

validation body · pagination · path-heavy search
empty list returned as 404 · DB migrations · mapping
soft delete · PATCH · logging · OpenAPI · Actuator
profiles · JWT security · Docker · Testcontainers

Phase 1 – API quality

Highest value. Low risk. Do this phase first.

1.1 Validation error handler

Why. @Valid failures still use Spring's default 400.

Clients need field messages in a stable JSON shape.

What. Teach `GlobalExceptionHandler` to catch

`MethodArgumentNotValidException`.

Match the 404 shape: timestamp, status, error, message, plus errors: { field: message }.

Also catch a generic `Exception` and return 500.

Never put a stack trace in the response body.

Touch. `GlobalExceptionHandler` + 400 controller tests.

package: `com.codemyth.exception`

1.2 Empty search → 200 OK + []

Why. An empty list is not a missing resource.

Use 404 only when that employee id does not exist (GET, PUT, or DELETE by id).

What. In `EmployeeService`, do not throw when a city, age, salary, or name lookup is empty.

Return an empty list instead.

Keep id-not-found as `EmployeeNotFoundException`.

Fix any leftover typo: "bu Salary" → "by Salary".

Touch. `EmployeeService`, plus the related tests.

1.3 Pagination on the list

Why. An unbounded `findAll()` will not scale.

What. New list call:

`GET /api/v1/employees?page=0&size=20&sort=empId,asc`

Use Spring Pageable. Response shows content, page, size, totalElements, totalPages.

(Spring Page JSON, or a small wrapper DTO.)

Default size = 20. Cap the size (example: 100).

Touch. Controller, Service, Repository, tests, README, and the employee-ui list load.

1.4 Unified search (query params)

Why. Split paths such as `/city/{x}` and `/age/{x}` cannot combine filters cleanly.

Chosen API:

`GET /api/v1/employees?name=&city=&age=&salary=`
`...plus page and size on that same URL.`

Filters are optional. Use `JpaSpecificationExecutor`, or a null-safe `@Query`.

Mark the old path search endpoints **deprecated** (Javadoc + README). Remove them later, after employee-ui has moved to query params.

Touch. controller, repository, service, tests, README, and employee-ui search.

Phase 2 – Schema, DTOs, mapping

2.1 Flyway migrations

Why. ddl-auto=none plus hand-run SQL drifts from one machine to the next.

What. Add flyway-core and flyway-mysql.

V1__create_employee.sql = today's employee table.

path: src/main/resources/db/migration/

Document Flyway in the README. Manual CREATE TABLE is no longer the main path. Keep a short note for databases that already exist.

2.2 Database indexes

Why. Search by city and by name gets faster.

What. Flyway file V2__indexes.sql

Index columns `emp_city` and `emp_name`.

2.3 DTO as a Java record

Why. Less boilerplate. The API contract stays immutable.

What. Turn `EmployeeRequest` and `EmployeeResponse` into records. Keep the validation annotations. Update the tests.

2.4 MapStruct mapper

Why. Hand-written entity ↔ DTO mapping in the service breaks easily as fields grow.

What. Add `EmployeeMapper` (MapStruct). The service only orchestrates. Delete private `mapToResponse`.

2.5 Soft delete

Why. A hard delete – especially delete-all – throws away data you may need back.

Chosen design.

- Column `deleted`, boolean, default false.
- Flyway script: `V3__soft_delete.sql`
- Hide deleted rows. Use `@SQLRestriction` ("`deleted = false`") or the same filter in queries.
- `DELETE` by id sets `deleted = true`. It does not remove the row.
- Later, optional: an admin hard-delete endpoint.

2.6 PATCH – partial update

Why. `PUT` makes the client send every field, even when only one value changed.

What.

`PATCH /api/v1/employees/{empId}`

- New type: `EmployeePatchRequest`.
- Every field is optional / nullable.
- Apply only the fields that are not null.

Phase 3 – Observability and docs

3.1 Logging

Why. Debug production without attaching a debugger.

What. `SLF4J` via `@Slf4j` on the service. Add a light log on the controller only if it helps.

Logging rules, continued:

- **Info** – create, update, delete. Log ids only.
- **Warn** – not-found paths.
- **Error** – unexpected failures.
- Do not log passwords or extra personal data.

3.2 springdoc-openapi

Why. People and frontends need interactive docs.

What. Add springdoc. Turn on Swagger UI.

Add short notes on the controller.

Link the UI path from the README

(Boot 4 / springdoc path, as configured).

3.3 Actuator

Why. Ops and Docker need a health check.

What. spring-boot-starter-actuator.

Expose **health** and **info** by default.

Lock this down further in production.

3.4 Profiles

Why. Dev and prod should not share one flat file forever.

What.

- application.properties – shared defaults
- application-dev.properties – local MySQL, and optional SQL logging
- application-prod.properties – stricter settings

Run: **--spring.profiles.active=dev**

Phase 4 – Security and ops

4.1 Spring Security + JWT

Why. Open write and delete routes are unsafe on any shared network. Especially DELETE /employees.

Chosen rules for this REST API:

- **Public** (friendly for dev): GET list, get, search, Swagger, and health.
- **Authenticated:** POST, PUT, PATCH, DELETE.

POST /api/v1/auth/login → issues a JWT

- An in-memory user is fine for v1.
Database users can come later.
- Keep CORS for ports 5173 and 3000, and allow the Authorization header.

4.2 Docker

Why. One command for a local stack, and a setup closer to real deployment.

What.

- Multi-stage Dockerfile: Maven build, then a JRE image.
- docker-compose.yml runs MySQL + the app.
- Pass DB_* values through the environment.
- Commit **.env.example** only. No real secrets in git.

4.3 Testcontainers

Why. Unit tests and WebMvc tests do not prove that Flyway plus a real MySQL actually work.

What. One integration test:

- MySQL in a container
- Flyway runs the migrations
- A few critical HTTP flows
- Keep the existing unit and WebMvc tests

Target layout, as phases land

EmployeeAPICRUD/

docs/ROADMAP.md

src/main/resources/

db/migration/ ← Phase 2

application.properties

application-dev.properties

application-prod.properties

Dockerfile ← Phase 4

docker-compose.yml

.env.example

Implementation order

1. Phase 1 – validation handler, empty list as [], pagination, query-param search + tests
2. Phase 2 – Flyway, indexes, records, MapStruct, soft delete, PATCH
3. Phase 3 – logging, OpenAPI, Actuator, profiles
4. Phase 4 – JWT security, Docker, Testcontainers
5. After every breaking phase, update employee-ui and README.md

Checklist – done already

- ✓ Phase 1.1 Validation error handler
- ✓ Phase 1.2 Empty search → 200 + []
- ✓ Phase 1.3 Pagination
- ✓ Phase 1.4 Query-param search (old paths marked deprecated)

Checklist – still to do

- ☐ 2.1 Flyway ☐ 2.2 Indexes
- ☐ 2.3 DTO records ☐ 2.4 MapStruct
- ☐ 2.5 Soft delete ☐ 2.6 PATCH

Checklist – still to do

- ☐ 3.1 Logging
- ☐ 3.2 springdoc-openapi
- ☐ 3.3 Actuator
- ☐ 3.4 Profiles
- ☐ 4.1 JWT security
- ☐ 4.2 Docker and Compose
- ☐ 4.3 Testcontainers

One-line memory

Phase 1 makes the API honest and pageable.

Phase 2 makes the schema and mapping solid.

Phase 3 makes the app visible and documented.

Phase 4 makes it safe to run and ship.

Source: [EmployeeAPICRUD/docs/ROADMAP.md](#)

These notes follow that roadmap. They do not replace it – tick the markdown checklist as each phase lands.